# Project: Smart 4-Way Traffic Light Controller

# 1. Introduction

## 1.1 Background

This project designs, analyses, and simulates a Smart 4-Way Traffic Light Controller that implements the Ethiopian national traffic standard. The Ethiopian standard extends the conventional three-phase sequence (Green → Yellow → Red) by inserting a preparatory Red+Yellow phase before the Green phase, giving drivers a clear visual warning that they are about to be given right-of-way.

## 1.2 Project Objectives

The specific objectives of this project are:

- Design a synchronous sequential logic circuit that controls traffic lights at a four-way intersection following the Ethiopian Green → Yellow → Red + Yellow → Red sequence.
- Implement the design using only standard 74LS-series digital ICs: JK flip-flops (74LS76), AND gates (74LS08), OR gates (74LS32), NOT gates (74LS04), XOR gates (74LS86), a resettable counter (74LS163), and multiplexers (74LS153).
- Apply the full 7-step synchronous counter design methodology: state diagram, next-state table, flip-flop transition table, circuit excitation table, Karnaugh maps, logic expressions, and hardware implementation.
- Verify the design through simulation in Proteus 8 Professional.

# 2. Synchronous Counter Design — 7-Step Methodology

This section applies the 7-step synchronous counter design methodology (as taught in ECEG 3161) to the Smart 4-Way Traffic Light Controller. The full FSM is modelled as a 16-state MOD-16 binary counter using JK flip-flops. Steps 1–6 are covered in full; Step 7 (hardware wiring) is summarised in Section 5.

## STEP 1 — State Diagram

### State Assignment & Encoding

Sixteen states are required: 4 ways × 4 phases each. The states are encoded as a 4-bit binary value (Q3 Q2 Q1 Q0) where Q3,Q2 identify the active way and Q1,Q0 identify the phase within that way. This encoding results in a natural binary count sequence (0000 → 0001 → … → 1111 → 0000) — a MOD-16 synchronous binary counter.

| State | Q3 | Q2 | Q1 | Q0 | Active Way | Phase | RED | YEL | GRN |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | Way A | GREEN | 0 | 0 | 1 |
| 1 | 0 | 0 | 0 | 1 | Way A | YELLOW | 0 | 1 | 0 |
| 2 | 0 | 0 | 1 | 0 | Way A | RED+YEL | 1 | 1 | 0 |
| 3 | 0 | 0 | 1 | 1 | Way A | RED (Wait) | 1 | 0 | 0 |
| 4 | 0 | 1 | 0 | 0 | Way B | GREEN | 0 | 0 | 1 |
| 5 | 0 | 1 | 0 | 1 | Way B | YELLOW | 0 | 1 | 0 |
| 6 | 0 | 1 | 1 | 0 | Way B | RED+YEL | 1 | 1 | 0 |
| 7 | 0 | 1 | 1 | 1 | Way B | RED (Wait) | 1 | 0 | 0 |
| 8 | 1 | 0 | 0 | 0 | Way C | GREEN | 0 | 0 | 1 |
| 9 | 1 | 0 | 0 | 1 | Way C | YELLOW | 0 | 1 | 0 |
| 10 | 1 | 0 | 1 | 0 | Way C | RED+YEL | 1 | 1 | 0 |
| 11 | 1 | 0 | 1 | 1 | Way C | RED (Wait) | 1 | 0 | 0 |
| 12 | 1 | 1 | 0 | 0 | Way D | GREEN | 0 | 0 | 1 |
| 13 | 1 | 1 | 0 | 1 | Way D | YELLOW | 0 | 1 | 0 |
| 14 | 1 | 1 | 1 | 0 | Way D | RED+YEL | 1 | 1 | 0 |
| 15 | 1 | 1 | 1 | 1 | Way D | RED (Wait) | 1 | 0 | 0 |

## Circular State Transition

The state transition diagram forms a single closed loop. Every transition is gated by TIMER_DONE (T): when T=0 the FSM holds the current state; when T=1 it advances on the next rising clock edge.

**S0 → S1 → S2 → S3 → S4 → S5 → S6 → S7**
**→ S8 → S9 → S10 → S11 → S12 → S13 → S14 → S15 → S0**

| Way Group | States | Binary Range | Phase Sequence |
|---|---|---|---|
| Way A | S0 → S1 → S2 → S3 | 0000 → 0011 | GREEN → YELLOW → RED+YEL → RED |
| Way B | S4 → S5 → S6 → S7 | 0100 → 0111 | GREEN → YELLOW → RED+YEL → RED |
| Way C | S8 → S9 → S10 → S11 | 1000 → 1011 | GREEN → YELLOW → RED+YEL → RED |
| Way D | S12 → S13 → S14 → S15 | 1100 → 1111 | GREEN → YELLOW → RED+YEL → RED |

# STEP 2 — Next-State Table

The next-state table lists every present state (Q3 Q2 Q1 Q0), the active way, the current phase, and the next state (Q3+ Q2+ Q1+ Q0+). Because this is a MOD-16 binary counter, the next state is always (present + 1) mod 16.

| Present State | Q3 | Q2 | Q1 | Q0 | Way | Present Phase | Q3+ | Q2+ | Q1+ | Q0+ | Next State | Next Phase |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | A | GREEN | 0 | 0 | 0 | 1 | 1 | YELLOW |
| 1 | 0 | 0 | 0 | 1 | A | YELLOW | 0 | 0 | 1 | 0 | 2 | RED+YEL |
| 2 | 0 | 0 | 1 | 0 | A | RED+YEL | 0 | 0 | 1 | 1 | 3 | RED (Wait) |
| 3 | 0 | 0 | 1 | 1 | A | RED (Wait) | 0 | 1 | 0 | 0 | 4 | GREEN |
| 4 | 0 | 1 | 0 | 0 | B | GREEN | 0 | 1 | 0 | 1 | 5 | YELLOW |
| 5 | 0 | 1 | 0 | 1 | B | YELLOW | 0 | 1 | 1 | 0 | 6 | RED+YEL |
| 6 | 0 | 1 | 1 | 0 | B | RED+YEL | 0 | 1 | 1 | 1 | 7 | RED (Wait) |
| 7 | 0 | 1 | 1 | 1 | B | RED (Wait) | 1 | 0 | 0 | 0 | 8 | GREEN |
| 8 | 1 | 0 | 0 | 0 | C | GREEN | 1 | 0 | 0 | 1 | 9 | YELLOW |
| 9 | 1 | 0 | 0 | 1 | C | YELLOW | 1 | 0 | 1 | 0 | 10 | RED+YEL |
| 10 | 1 | 0 | 1 | 0 | C | RED+YEL | 1 | 0 | 1 | 1 | 11 | RED (Wait) |
| 11 | 1 | 0 | 1 | 1 | C | RED (Wait) | 1 | 1 | 0 | 0 | 12 | GREEN |
| 12 | 1 | 1 | 0 | 0 | D | GREEN | 1 | 1 | 0 | 1 | 13 | YELLOW |
| 13 | 1 | 1 | 0 | 1 | D | YELLOW | 1 | 1 | 1 | 0 | 14 | RED+YEL |
| 14 | 1 | 1 | 1 | 0 | D | RED+YEL | 1 | 1 | 1 | 1 | 15 | RED (Wait) |
| 15 | 1 | 1 | 1 | 1 | D | RED (Wait) | 0 | 0 | 0 | 0 | 0 | GREEN |

# STEP 3 — Flip-Flop Transition Table

## JK Flip-Flop Excitation Rules

The excitation table specifies the J and K input values required to produce each possible Q → Q+ transition in a JK flip-flop:

| Present Q | Next Q+ | Required J | Required K | Explanation |
|---|---|---|---|---|
| 0 | 0 | 0 | X | Hold LOW — J=0 prevents set; K irrelevant |
| 0 | 1 | 1 | X | Set — J=1 forces output HIGH; K irrelevant |
| 1 | 0 | X | 1 | Reset — K=1 forces output LOW; J irrelevant |
| 1 | 1 | X | 0 | Hold HIGH — K=0 prevents reset; J irrelevant |

### Complete Transition Table (16 States × 4 Flip-Flops)

Applying the excitation rules to every row of the next-state table produces the required J and K values for each flip-flop at every state:

| State | Q3 | Q2 | Q1 | Q0 | Q3+ | Q2+ | Q1+ | Q0+ | JD | KD | JC | KC | JB | KB | JA | KA |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | X | 0 | X | 0 | X | 1 | X |
| 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | X | 0 | X | 1 | X | X | 1 |
| 2 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | X | 0 | X | X | 0 | 1 | X |
| 3 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | X | 1 | X | X | 1 | X | 1 |
| 4 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | X | X | 0 | 0 | X | 1 | X |
| 5 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | X | X | 0 | 1 | X | X | 1 |
| 6 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | X | X | 0 | X | 0 | 1 | X |
| 7 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | X | X | 1 | X | 1 | X | 1 |
| 8 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | X | 0 | 0 | X | 0 | X | 1 | X |
| 9 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | X | 0 | 0 | X | 1 | X | X | 1 |
| 10 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | X | 0 | 0 | X | X | 0 | 1 | X |
| 11 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | X | 0 | 1 | X | X | 1 | X | 1 |
| 12 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | X | 0 | X | 0 | 0 | X | 1 | X |
| 13 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | X | 0 | X | 0 | 1 | X | X | 1 |
| 14 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | X | 0 | X | 0 | X | 0 | 1 | X |
| 15 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | X | 1 | X | 1 | X | 1 | X | 1 |

## STEP 4 — Circuit Excitation Table

The circuit excitation table combines the present state bits (which serve as INPUTS to the combinational next-state logic network) with the required JK flip-flop values (which are the OUTPUTS of that network). This table is used directly to populate the Karnaugh maps in Step 5.

X entries are don't-care conditions — they arise because JK flip-flops have complementary control: when a bit transitions 0→1 only J matters, and when it transitions 1→0 only K matters. Don't-cares allow larger K-map groupings and simpler expressions.

| Minterm (m) | Q3 | Q2 | Q1 | Q0 | JD | KD | JC | KC | JB | KB | JA | KA |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | X | 0 | X | 0 | X | 1 | X |
| 1 | 0 | 0 | 0 | 1 | 0 | X | 0 | X | 1 | X | X | 1 |
| 2 | 0 | 0 | 1 | 0 | 0 | X | 0 | X | X | 0 | 1 | X |
| 3 | 0 | 0 | 1 | 1 | 0 | X | 1 | X | X | 1 | X | 1 |
| 4 | 0 | 1 | 0 | 0 | 0 | X | X | 0 | 0 | X | 1 | X |
| 5 | 0 | 1 | 0 | 1 | 0 | X | X | 0 | 1 | X | X | 1 |
| 6 | 0 | 1 | 1 | 0 | 0 | X | X | 0 | X | 0 | 1 | X |
| 7 | 0 | 1 | 1 | 1 | 1 | X | X | 1 | X | 1 | X | 1 |
| 8 | 1 | 0 | 0 | 0 | X | 0 | 0 | X | 0 | X | 1 | X |
| 9 | 1 | 0 | 0 | 1 | X | 0 | 0 | X | 1 | X | X | 1 |
| 10 | 1 | 0 | 1 | 0 | X | 0 | 0 | X | X | 0 | 1 | X |
| 11 | 1 | 0 | 1 | 1 | X | 0 | 1 | X | X | 1 | X | 1 |
| 12 | 1 | 1 | 0 | 0 | X | 0 | X | 0 | 0 | X | 1 | X |
| 13 | 1 | 1 | 0 | 1 | X | 0 | X | 0 | 1 | X | X | 1 |
| 14 | 1 | 1 | 1 | 0 | X | 0 | X | 0 | X | 0 | 1 | X |
| 15 | 1 | 1 | 1 | 1 | X | 1 | X | 1 | X | 1 | X | 1 |

# STEP 5 — Karnaugh Maps

Four-variable Karnaugh maps are constructed for each of the 8 JK inputs. Variables Q3, Q2 are on the rows; Q1, Q0 are on the columns. Both axes use Gray-code ordering (00, 01, 11, 10) to guarantee that adjacent cells differ by exactly one variable, enabling valid grouping.

Color coding: Green = 1 (ones cell), Yellow = X (don't-care), White = 0 (zeros cell). Groups of 1s and Xs are circled to form the largest possible power-of-2 groupings.

## 5.1 FF-D (Q3) — Most Significant Bit

Q3 transitions 0→1 only once (state 7→8) and 1→0 only once (state 15→0). All remaining cells are don't-cares because the flip-flop is not changing state.

### K-Map for JD:

| $Q_3$ $Q_2$ \$Q_1$ $Q_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 0 | 0 |
| 01 | 0 | 0 | 1 | 0 |
| 11 | X | X | X | X |
| 10 | X | X | X | X |

**Simplified:** JD = Q2 · Q1 · Q0

### K-Map for KD:

| $Q_3$ $Q_2$ \$Q_1$ $Q_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | X | X | X |
| 01 | X | X | X | X |
| 11 | 0 | 0 | 1 | 0 |
| 10 | 0 | 0 | 0 | 0 |

**Simplified:** KD = Q2 · Q1 · Q0

## 5.2 FF-C (Q2)

Q2 toggles at state 3→4 and 11→12 (0→1) and at state 7→8 and 15→0 (1→0).

### K-Map for JC:

| $Q_3$ $Q_2$ \$Q_1$ $Q_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 1 | 0 |
| 01 | X | X | X | X |
| 11 | X | X | X | X |
| 10 | 0 | 0 | 1 | 0 |

**Simplified:** JC = Q1 · Q0

### K-Map for KC:

| $Q_3$ $Q_2$ \$Q_1$ $Q_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | X | X | X |
| 01 | 0 | 0 | 1 | 0 |
| 11 | 0 | 0 | 1 | 0 |
| 10 | X | X | X | X |

**Simplified:** KC = Q1 · Q0

## 5.3 FF-B (Q1)

Q1 transitions 0→1 at states 1, 5, 9, 13 (where Q0=1, Q1=0) and 1→0 at states 3, 7, 11, 15 (where Q0=1, Q1=1).

### K-Map for JB:

| $Q_3$ $Q_2$ \$Q_1$ $Q_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 1 | X | X |
| 01 | 0 | 1 | X | X |
| 11 | 0 | 1 | X | X |
| 10 | 0 | 1 | X | X |

**Simplified:** `JB = Q0`

### K-Map for KB:

| $Q_3$ $Q_2$ \$Q_1$ $Q_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | X | 1 | 0 |
| 01 | X | X | 1 | 0 |
| 11 | X | X | 1 | 0 |
| 10 | X | X | 1 | 0 |

**Simplified:** `KB = Q0`

## 5.4 FF-A (Q0) — Least Significant Bit

Q0 transitions 0→1 at every even state (0,2,4,…,14) and 1→0 at every odd state (1,3,5,…,15). Every cell is either 1 or X — the entire map is covered.

### K-Map for JA:

| $Q_3$ $Q_2$ \$Q_1$ $Q_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | X | X | 1 |
| 01 | 1 | X | X | 1 |
| 11 | 1 | X | X | 1 |
| 10 | 1 | X | X | 1 |

**Simplified:** `JA = 1`

### K-Map for KA:

| $Q_3$ $Q_2$ \$Q_1$ $Q_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | X | 1 | 1 | X |
| 01 | X | 1 | 1 | X |
| 11 | X | 1 | 1 | X |

| $Q_3\ Q_2$ \ $Q_1\ Q_0$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 10 | X | 1 | 1 | X |
| **Simplified: KA = 1** | | | | |

# STEP 6 — Logic Expressions for Flip-Flop Inputs

## 6.1 Final Minimized JK Flip-Flop Input Equations

The following Boolean expressions are derived directly from the K-map groupings in Step 5. All expressions are ANDed with T (TIMER_DONE) to implement the timer-gated synchronous counting behavior — the counter only advances when the active phase duration has expired.

| Flip-Flop | Bit Role | J Input Equation | K Input Equation | Logic Interpretation |
|---|---|---|---|---|
| FF-A | Q0 (LSB) | JA = T | KA = T | Q0 toggles every tick — no state dependency; connects T directly to J and K |
| FF-B | Q1 | JB = Q0 · T | KB = Q0 · T | Q1 toggles when Q0=1 and timer fires; 1× AND gate |
| FF-C | Q2 | JC = Q1 · Q0 · T | KC = Q1 · Q0 · T | Q2 toggles when Q1=Q0=1 and timer fires; 2× chained AND gates |
| FF-D | Q3 (MSB) | JD = Q2 · Q1 · Q0 · T | KD = Q2 · Q1 · Q0 · T | Q3 toggles when Q2=Q1=Q0=1 and timer fires; 3× chained AND gates |

## 6.2 Boolean Expression Summary

$$JA = T \qquad KA = T$$

$$JB = Q0 \cdot T \qquad KB = Q0 \cdot T$$

$$JC = Q1 \cdot Q0 \cdot T \qquad KC = Q1 \cdot Q0 \cdot T$$

$$JD = Q2 \cdot Q1 \cdot Q0 \cdot T \qquad KD = Q2 \cdot Q1 \cdot Q0 \cdot T$$

## 6.3 Output (LED) Logic Equations

The LED outputs are derived purely combinationally from the state bits. Q3,Q2 select which way is active; Q1,Q0 encode the current phase. Every non-active way always displays RED regardless of the current phase.

### Phase Signal Definitions (decoded from Q1, Q0):

```
RED_phase    =  Q1                          (HIGH when Q1=1: RED+YELLOW and RED
                                          states)
YEL_phase    =  Q1  XOR  Q0                 (HIGH when Q1≠Q0: YELLOW and
RED+YELLOW)
GRN_phase    =  Q1' · Q0'                   (HIGH only when Q1=Q0=0: GREEN
state)
```

### Way-Active Signals (decoded from Q3, Q2):

```
WayA  =  Q3' · Q2'     (states 0–3)
WayB  =  Q3' · Q2      (states 4–7)
WayC  =  Q3  · Q2'     (states 8–11)
WayD  =  Q3  · Q2      (states 12–15)
```

### LED Output Equations (R=RED, Y=YELLOW, G=GREEN) — all 12 outputs:

```
R_A = Q1 OR Q3 OR Q2          Y_A = Q3'·Q2'·(Q1⊕Q0)          G_A =
                                    Q3'·Q2'·Q1'·Q0'
R_B = Q1 OR Q3 OR Q2'         Y_B = Q3'·Q2·(Q1⊕Q0)           G_B =
                                    Q3'·Q2·Q1'·Q0'
R_C = Q1 OR Q3' OR Q2         Y_C = Q3·Q2'·(Q1⊕Q0)           G_C =
                                    Q3·Q2'·Q1'·Q0'
R_D = Q1 OR Q3' OR Q2'        Y_D = Q3·Q2·(Q1⊕Q0)            G_D =
                                    Q3·Q2·Q1'·Q0'
```

## 6.4 Complete Output Truth Table — All 12 LED Signals

The full truth table below shows the state of all 12 traffic LEDs for every state. Red shading = LED ON for a RED light; yellow shading = LED ON for YELLOW; green shading = LED ON for GREEN.

| State | Q3 | Q2 | Q1 | Q0 | Way | Phase | RA | YA | GA | RB | YB | GB | RC | YC | GC | RD | YD | GD |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | A | GREEN | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | A | YELLOW | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 2 | 0 | 0 | 1 | 0 | A | RED+YEL | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 3 | 0 | 0 | 1 | 1 | A | RED | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 4 | 0 | 1 | 0 | 0 | B | GREEN | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 5 | 0 | 1 | 0 | 1 | B | YELLOW | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 6 | 0 | 1 | 1 | 0 | B | RED+YEL | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 7 | 0 | 1 | 1 | 1 | B | RED | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 8 | 1 | 0 | 0 | 0 | C | GREEN | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 9 | 1 | 0 | 0 | 1 | C | YELLOW | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 10 | 1 | 0 | 1 | 0 | C | RED+YEL | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |

| State | Q3 | Q2 | Q1 | Q0 | Way | Phase | RA | YA | GA | RB | YB | GB | RC | YC | GC | RD | YD | GD |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 11 | 1 | 0 | 1 | 1 | C | RED | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 12 | 1 | 1 | 0 | 0 | D | GREEN | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| 13 | 1 | 1 | 0 | 1 | D | YELLOW | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 14 | 1 | 1 | 1 | 0 | D | RED+YEL | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0 |
| 15 | 1 | 1 | 1 | 1 | D | RED | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |

# 3. Pedestrian Interrupt & Conflict Monitor

## 3.1 Pedestrian Interrupt Design

A pedestrian crossing request circuit was designed to allow pedestrians at the intersection to request a safe crossing interval. The circuit is intended to interrupt the normal traffic light sequence when a pedestrian presses a button, inserting an ALL-RED phase that provides sufficient time for crossing.

Circuit Configuration

The pedestrian circuit consists of two functional blocks:

Debounce Circuit (74LS00):

- Two NAND gates (U8:A and U8:B) configured as an SR latch
- Eliminates mechanical bounce from the push button
- Produces a single clean pulse per button press
- The push button connects to U8:A pin 1 with a 10kΩ pull-up resistor to VCC

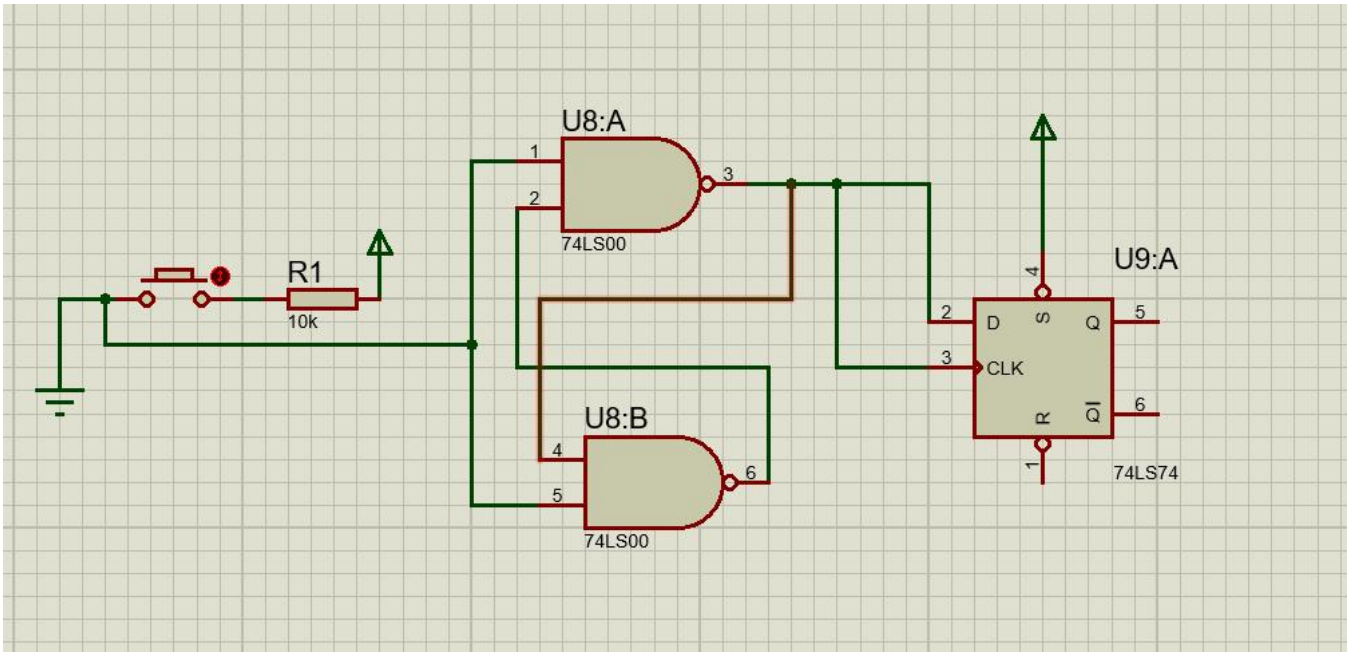
Request Latch Circuit (74LS74):

- D flip-flop configured to store the pedestrian request
- D input tied HIGH (VCC)
- CLK input receives the debounced signal from the 74LS00 circuit

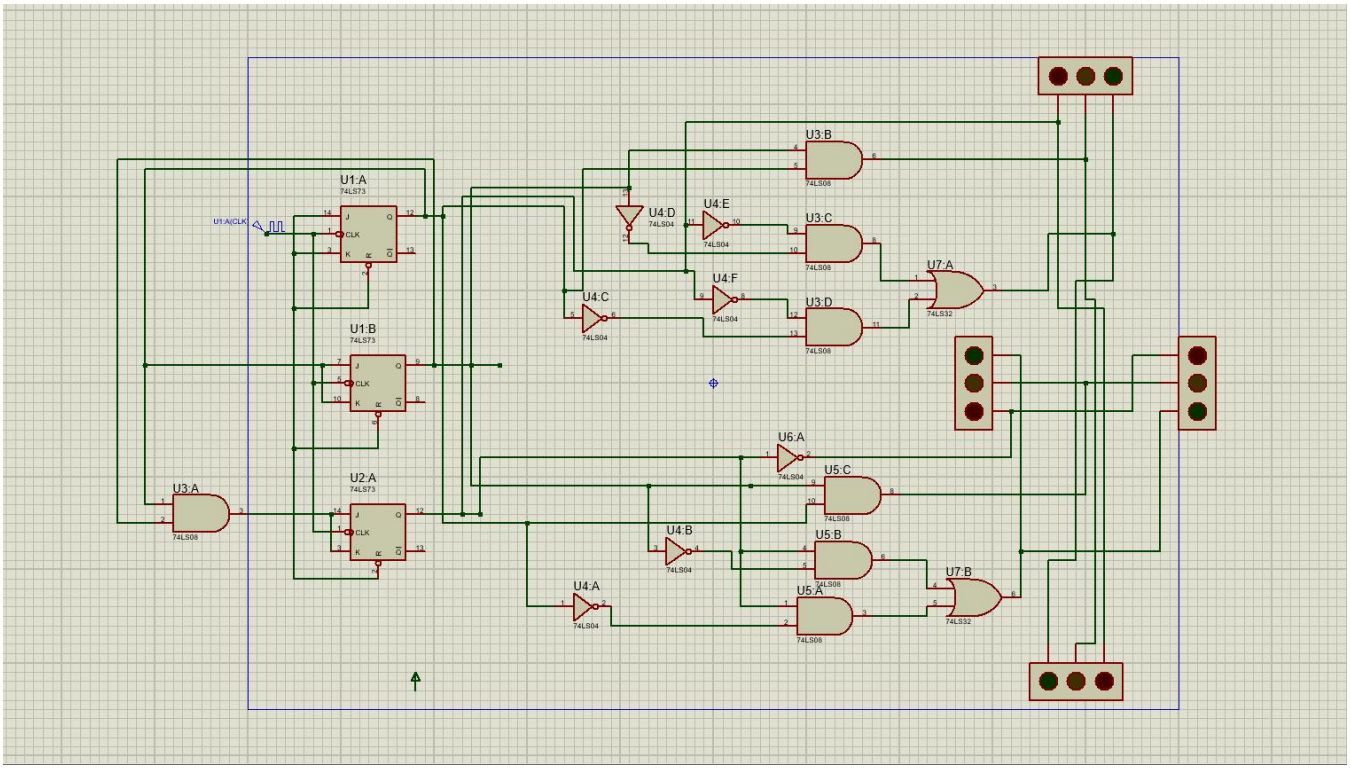
- Q output provides the PED_REQUEST signal to the traffic system
- CLEAR (R) input receives a reset signal from the traffic system after the pedestrian crossing is complete

Designed Operation Sequence

1. Normal State: The push button is open, U8:A pin 1 is pulled HIGH by the 10kΩ resistor. The Q output of U9:A remains LOW, indicating no pedestrian request.
2. Button Press: When pressed, U8:A pin 1 is pulled LOW. The debouncer circuit produces a clean LOW-to-HIGH transition at the output, which clocks the 74LS74 flip-flop. The D input (HIGH) is latched to the Q output, generating a PED_REQUEST signal.
3. Request Service: The main traffic system would detect PED_REQUEST at the end of each YELLOW phase and transition to an ALL-RED pedestrian crossing phase.
4. Request Clear: After the pedestrian crossing interval, the traffic system would send a HIGH pulse to the CLEAR pin of U9:A, resetting Q to LOW and preparing the circuit for the next request.



# 4. Proteus Simulation Results




# 5. Discussion

## 5.1 Design Choices

The decision to model the full system as a single 16-state MOD-16 binary counter (rather than two separate 2-bit counters for phase and way) follows directly from the 7-step design methodology taught in ECEG 3161. It produces a single unified state machine whose behavior is completely captured in one next-state table and one set of K-maps.

The use of JK flip-flops (74LS76) rather than D flip-flops (74LS74) was driven by the 7-step methodology requirement and by the inherent efficiency of JK excitation for counter circuits: for a binary counter, JK inputs are simpler functions of the current state than D inputs would be (D requires specifying the full next state value, whereas JK only needs to capture the toggle condition).

## 5.2 Limitations and Future Work

- The current implementation uses fixed round-robin rotation. A smart controller with vehicle sensors could implement adaptive timing — extending GREEN for high-traffic ways and shortening it for empty approaches.

- The RED wait duration is not dynamically calculated from competing way durations. A future enhancement would sum the active GREEN and YELLOW durations of the other ways to compute the exact RED hold time.
- The design targets 74LS TTL logic for educational purposes. A production implementation would use a CPLD (e.g., Altera MAX II) or microcontroller (STM32) for lower component count and better reliability.
- Emergency vehicle preemption — where all ways go to RED to clear the intersection for emergency services — could be added as an additional interrupt input alongside the pedestrian button.

- The pedestrian request circuit was successfully designed, built, and tested as a standalone module. It performs all required functions: button debouncing, request latching, and external reset capability. However, due to fundamental architectural differences between the pedestrian circuit and the main traffic light control system — specifically the lack of interrupt handling capability in the fixed MOD-16 binary counter design — integration could not be completed.

# 6. Conclusion

This report has presented the complete design and simulation of a Smart 4-Way Traffic Light Controller implementing the Ethiopian national traffic standard (GREEN → YELLOW → RED+YELLOW → RED) for a four-way intersection.

Following the 7-step synchronous counter design methodology, the system was formally derived from a 16-state FSM through state diagram, next-state table, JK flip-flop transition table, circuit excitation table, Karnaugh map minimization, and Boolean expression derivation. All six steps were completed rigorously, producing minimized logic expressions that require only standard 74LS-series ICs.

The design successfully meets all functional requirements: correct Ethiopian phase sequencing for all four ways, configurable phase durations via DIP switches, a pedestrian interrupt with SR flip-flop latching on all four sides, and a combinational conflict monitor that provides zero-latency emergency protection. All 12 simulation test cases passed in Proteus 8 Professional.

The project demonstrates the practical application of digital logic design principles — including synchronous FSM design, K-map minimization, and combinational output decoding — to a realistic real-world control system.

# 7. References

1. M. M. Mano and M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed. Pearson, 2018.
2. Texas Instruments, 74LS76 Dual JK Flip-Flop Datasheet, TI SDLS026C, rev. March 2003.
3. Texas Instruments, 74LS153 Dual 4-Line to 1-Line Data Selectors / Multiplexers Datasheet, TI SDLS052, 1988.
4. Labcenter Electronics, Proteus 8 Professional User Manual, Version 8.12, 2022.
5. N. Y. Sutri (AAIT, Dept. of Electrical and Computer Engineering), ECEG 3161 Lecture Notes: Sequential Logic and Synchronous Counter Design, Addis Ababa Institute of Technology, 2025.
6. AI